1. What is Unit Testing? Explain its purpose.

Unit Testing is a software testing technique where individual components or modules of a program are tested separately to verify that each part works correctly.
Purpose: Its main goal is to detect early bugs and ensure that each unit performs as designed before integrating with other components. It helps improve code reliability and maintainability.

---

2. Define Software Maintenance. Mention its types.

Software Maintenance is the process of modifying and updating software applications after deployment to correct faults, improve performance, or adapt to a changed environment.
Types:

1. **Corrective Maintenance** – Fixing bugs or errors found after release.
2. **Adaptive Maintenance** – Adjusting the system to new environments or requirements.
3. **Perfective Maintenance** – Enhancing performance or adding new features.
4. **Preventive Maintenance** – Making changes to prevent future issues.

---

3. Define Project Maintenance. Why is maintenance required after software deployment?

Project Maintenance refers to the set of activities performed to keep software operational, reliable, and up to date after it has been delivered to users.
Need for maintenance: It is required to fix post-deployment defects, adapt to new technologies or user needs, improve efficiency, and ensure smooth, long-term system performance.

---

4. What is the Software Maintenance Life Cycle (SMLC)? List its key phases.

The Software Maintenance Life Cycle (SMLC) defines the stages through which software passes during its maintenance phase.
Key Phases:

1. Identification and Tracing of problems.
2. Analysis of required modifications.
3. Design of the changes.
4. Implementation and testing.

5. System release and acceptance.
6. Post-release review.

---

5. What is Transaction Flow Testing? Give a simple example.

Transaction Flow Testing is a white-box testing technique that focuses on logical paths of transactions through a program. It helps verify that all transaction paths execute correctly.
Example: In an online shopping system, a transaction may include "Login → Select Item → Add to Cart → Checkout." Each of these steps represents a transaction flow tested for correctness.

6. What is Web-Based Testing? Explain challenges involved in it.

Web-Based Testing is the process of testing web applications for functionality, usability, performance, compatibility, and security over the internet. It ensures that a web application runs correctly on different browsers, operating systems, and devices.
Challenges:

- **Browser Compatibility:** Different browsers may interpret HTML, CSS, or JavaScript differently.
- **Security Issues:** Web apps are prone to hacking, data theft, and unauthorized access.
- **Performance Testing:** Ensuring fast response time under heavy user load.
- **Network Dependency:** Slow or unstable internet can affect testing accuracy.

---

7. Explain the concept of Software Reuse and its advantages.

Software Reuse is the process of using existing software components, modules, or code in new applications instead of developing everything from scratch. It increases efficiency and reduces development cost.
Advantages:

- **Time Saving:** Reusing tested components speeds up development.
- **Cost Reduction:** Less coding effort reduces project cost.
- **Improved Quality:** Reused components are already tested and reliable.
- **Consistency:** Ensures uniform standards and functionality across applications.

---

8. Write a note on Benchmarking and Certification in software quality.

Benchmarking in software quality means comparing an organization's software processes or products with industry best standards to identify improvement areas.
Certification is a formal recognition by an authorized body that an organization's software development process meets specific quality standards such as **ISO 9001** or **CMMI**.
Together, benchmarking and certification help maintain consistent quality, gain customer trust, and improve competitiveness in the market.

---

9. Explain the importance of Coverage vs. Usage-based Testing.

Coverage-based Testing ensures that all parts of the code, such as statements, conditions, and paths, are tested at least once. It focuses on internal code structure.
Usage-based Testing focuses on how end-users actually use the system and tests the most frequently used functions first.
Importance: Coverage-based testing ensures code completeness, while usage-based testing improves reliability and user satisfaction by focusing on real-world scenarios. Using both methods together gives better software quality and testing efficiency.

10. Discuss different types of Software Testing: Unit Testing, Integration Testing, and System Testing.

Software testing is performed at various levels to ensure the software works as expected.

1. **Unit Testing:** It involves testing individual components or functions of a program to verify they work correctly in isolation. It is usually done by developers.
2. **Integration Testing:** After unit testing, modules are combined and tested as a group to verify that they interact correctly and data flows smoothly between them.
3. **System Testing:** This is the final testing stage where the complete integrated system is tested as a whole to ensure it meets specified requirements. It checks functionality, performance, and reliability before delivery.
   Each level helps in early defect detection, reducing cost and improving overall software quality.

---

11. Describe the process and methodology of Test Case Generation. Provide an example of a test case format.

Test Case Generation is the process of creating a set of test cases that can validate whether software functions correctly according to its requirements.
Process:

1. **Requirement Analysis:** Understand the functional and non-functional requirements.
2. **Test Scenario Identification:** Identify all possible input and output conditions.

3. **Designing Test Cases:** Define input data, expected results, and execution steps.
4. **Review and Validation:** Test cases are reviewed to ensure completeness.

   Example Format:
Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC01	Login with valid credentials	username, password	Redirect to dashboard	–	Pass/Fail
   This structured approach ensures that all requirements are covered and testing is traceable.

---

12. Explain Control Flow Testing and Data Flow Testing with examples.

Control Flow Testing focuses on the execution paths and logical flow of a program. It ensures that all decisions and branches are tested at least once.
Example: In an if-else statement, both true and false conditions must be tested to verify correct control flow.

Data Flow Testing focuses on the points where variables receive values and where those values are used in the program. It aims to detect anomalies like uninitialized variables or unused data.
Example: If a variable x is assigned a value but never used later, data flow testing identifies this as a potential issue.

Both methods are white-box techniques used to ensure logical correctness and data integrity in code.